



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 3

Lógica Fuzzy

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

Nas questões 1e, 2 e 4 as contas e os gráficos saíram do script `solucao.py` (Apêndice A). Nele implementamos as funções de pertinência triangular e trapezoidal, a inferência de Mamdani (mínimo para o E, máximo para juntar as regras) e a defuzzificação pelo centroide, e rodamos os dois sistemas com alguns valores de entrada.

Questão 1

a. Explique a diferença entre o tratamento de incerteza baseado em probabilidade e o tratamento de incerteza baseado em lógica fuzzy.

Resposta. As duas lidam com coisas diferentes. A probabilidade trata a incerteza sobre se um evento bem definido vai acontecer. “70% de chance de chuva” quer dizer que vai chover ou não vai, e ainda não sabemos qual. Depois que o evento acontece, a incerteza some e o valor vira 0 ou 1.

A lógica fuzzy trata conceitos sem fronteira definida, como “quente” ou “alto”. Dizer que 28 °C é quente com grau 0,7 não fala de chance: a temperatura já é conhecida, e o 0,7 diz o quanto ela se encaixa no conceito “quente”. Esse valor não muda depois de observado. Na probabilidade os eventos de uma partição somam 1. Na fuzzy um mesmo valor pode ter grau 0,7 em “quente” e 0,3 em “agradável” sem nenhuma exigência de soma.

b. O que é uma variável linguística? Dê um exemplo de duas e defina o respectivo conjunto de termos de cada.

Resposta. É uma variável cujos valores são palavras em vez de números, e cada palavra é um conjunto fuzzy sobre uma escala numérica. Exemplos:

- *velocidade*, na escala de 0 a 120 km/h, com os termos {lenta, média, rápida};
- *idade*, de 0 a 100 anos, com os termos {criança, jovem, adulto, idoso}.

c. Quais são os principais componentes que formam um sistema especialista fuzzy? Explique rapidamente do que se trata cada um deles.

Resposta.

- **Fuzzificador:** transforma o valor numérico de cada entrada nos graus de pertinência dos seus conjuntos.

- **Base de regras:** as regras do tipo “se ... então ...” escritas pelo especialista, que ligam os termos das entradas aos da saída.
- **Base de dados:** a definição das variáveis, dos seus universos e das funções de pertinência de cada termo.
- **Máquina de inferência:** calcula o quanto cada regra dispara e combina as conclusões num conjunto fuzzy de saída.
- **Defuzzificador:** converte esse conjunto de saída num número que pode ser usado como resposta ou comando.

d. Explique quais são as fases de um sistema fuzzy e do que se trata cada delas.

Resposta. São três, como nos slides da disciplina. Na **fuzzificação** definimos as variáveis de entrada e saída, os termos e a forma das funções de pertinência, e cada entrada numérica vira um grau em cada termo. Na **inferência** as regras são avaliadas em paralelo: o grau de cada condição é combinado (mínimo para E, máximo para OU), o consequente de cada regra é cortado nesse grau e as saídas de todas as regras são juntadas pelo máximo. Na **defuzzificação** a região resultante é convertida num valor da variável de saída, por exemplo pelo centroide ou pelo primeiro máximo.

e. Seja Universo = {0,1,2,3,4,5,6,7,9}. Sejam A e B dois subconjuntos nebulosos de Universo dado pelas funções de pertinências abaixo, determine $A \vee B$ e $A \wedge B$.

Resposta. A união usa o máximo e a interseção usa o mínimo, ponto a ponto. As tabelas da lista vão de 0 a 9, incluindo o 8.

u	0	1	2	3	4	5	6	7	8	9
A(u)	0	0	0,1	0,2	0,3	0,8	0,9	1	1	1
B(u)	1	1	0,9	0,8	0,7	0,5	0,4	0,2	0,2	0
$A \vee B = \max$	1	1	0,9	0,8	0,7	0,8	0,9	1	1	1
$A \wedge B = \min$	0	0	0,1	0,2	0,3	0,5	0,4	0,2	0,2	0

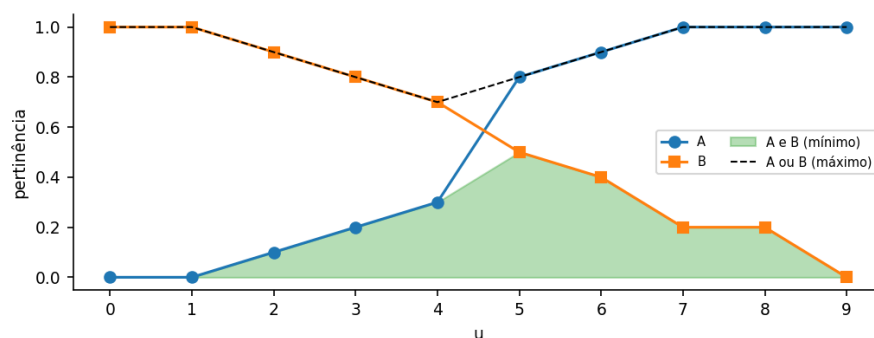


Figura 1. A, B, a união (tracejada) e a interseção (área verde).

A interseção nunca passa de 0,5, que é onde as duas curvas se cruzam, entre 4 e 5. Nenhum elemento pertence muito aos dois conjuntos ao mesmo tempo, o que era de se esperar, já que A cresce e B decresce.

Questão 2

Seja um sistema difuso para prever o número de turistas visitando um resort. Variáveis de entrada: temperatura (em graus Celsius) e luz do sol (expressa em uma porcentagem do máximo esperado de luz do sol). Saída: quantidade estimada de turistas (expressa em porcentagem da capacidade do resort). a) Para cada uma das variáveis linguísticas, defina três valores nebulosos. b) Represente em um gráfico (Fuzzyficação) os valores das respectivas variáveis linguísticas. c) Crie cinco regras relacionando as variáveis linguísticas acima.

Resposta. a) Os termos escolhidos foram:

- Temperatura (0 a 40 °C): fria (trapézio 0, 0, 10, 18), agradável (triângulo 15, 23, 30) e quente (trapézio 27, 32, 40, 40);
- Luz do sol (0 a 100%): nublado (0, 0, 20, 40), parcial (30, 50, 70) e ensolarado (60, 80, 100, 100);
- Turistas (0 a 100% da capacidade): poucos (0, 0, 20, 40), médio (30, 50, 70) e muitos (60, 80, 100, 100).

Nas pontas usamos trapézios, porque abaixo de 10 °C já é tão frio quanto possível para quem vai a um resort, e o grau não precisa diminuir. No meio, triângulos, com um valor típico (23 °C) e queda dos dois lados. Os conjuntos vizinhos se sobrepõem, para uma mesma temperatura poder ser um pouco agradável e um pouco quente.

b)

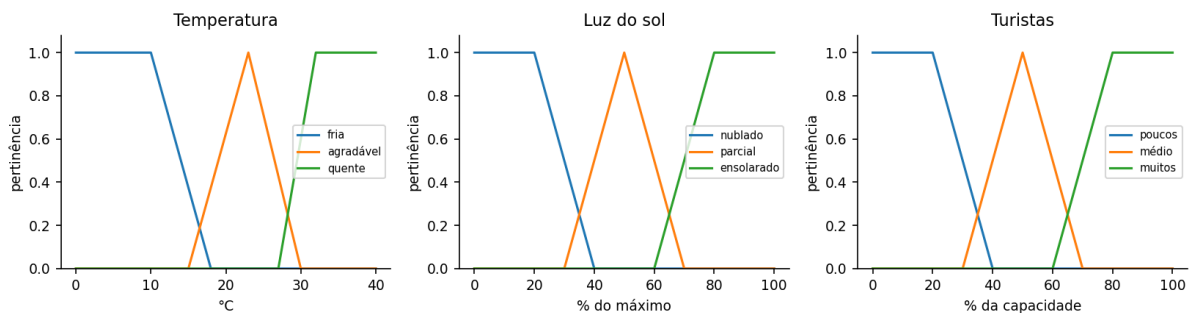


Figura 2. Funções de pertinência das três variáveis.

c) Regras:

- R1: Se a temperatura é quente e o sol é ensolarado, então turistas é muitos.
- R2: Se a temperatura é agradável e o sol é ensolarado, então turistas é muitos.
- R3: Se a temperatura é agradável e o sol é parcial, então turistas é médio.
- R4: Se a temperatura é fria, então turistas é poucos.
- R5: Se o sol é nublado, então turistas é poucos.

Para ver se as regras fazem sentido, rodamos o sistema no script. Com 28 °C e 65% de sol, a temperatura é agradável com grau 0,29 e quente com grau 0,20, e o sol é parcial com 0,25 e ensolarado com 0,25. Disparam R1 (0,20), R2 (0,25) e R3 (0,25), e o centroide dá 66,3% da capacidade. Com 33 °C e 90% de sol só R1 dispara, com grau 1, e a previsão sobe para 84,5%. Com 12 °C só R4 dispara e a previsão cai para 16,5%.

Questão 3

Considere as regras abaixo e identifique: Se atleta é alto e magro, então joga basquete; Se atleta é alto e gordo, então joga baralho; Se atleta é baixo e magro, então joga pingpong; Se atleta é baixo e gordo, então joga baralho. a) As variáveis linguísticas; b) Os valores de cada variável.

Resposta. a) São três: *altura* e *peso* (ou porte físico) do atleta, que são as entradas, e *esporte*, que é a saída.

b) Altura: {alto, baixo}. Peso: {magro, gordo}. Esporte: {basquete, baralho, pingpong}.

Questão 4

Supõe que uma universidade moderna resolveu mudar o seu método de avaliação, por pensar que o método em que um aluno com média 6,0 é aprovado e um aluno com média 5,9 é reprovado é um método injusto. [...] formule um sistema fuzzy que leve em consideração estes méritos, e dê como resposta se o aluno deve ser aprovado, reprovado ou fazer um curso de férias. Defina: a. os conjuntos fuzzy das entradas média_anual, frequência e participação, justificando a forma da função; b. os conjuntos da saída situação_aluno; c. as regras; d. o critério para o nível de pertinência do consequente quando uma regra com mais de uma entrada disparar; e. o método de defuzzificação.

Resposta. a) Entradas.

- Média anual (0 a 10): insuficiente (trapézio 0, 0, 4, 5,5), regular (triângulo 4,5, 6, 7,5) e boa (trapézio 6,5, 8, 10, 10).
- Frequência (0 a 100%): baixa (0, 0, 65, 75), regular (70, 80, 90) e alta (85, 92, 100, 100).
- Participação em aula (nota de 0 a 10 dada pelo professor): pouca (0, 0, 3, 5), moderada (3, 5,5, 8) e ativa (6, 8, 10, 10).

Nos termos das pontas usamos trapézios. Uma média 2 e uma média 3 são igualmente insuficientes, e uma frequência de 50% já é baixa por completo, então o grau fica em 1 numa faixa inteira. No termo do meio usamos triângulos com o pico no valor típico (média 6, que era o antigo corte, e frequência 80%) e queda linear dos dois lados. Os conjuntos vizinhos se sobrepõem, e isso é o que tira o salto do 5,9 para o 6,0: perto do antigo corte o aluno é, ao mesmo tempo, um pouco insuficiente e bastante regular. A frequência baixa termina em 75% porque esse é o mínimo exigido pela universidade. A gaussiana também funcionaria, mas as formas lineares são mais fáceis de explicar para quem vai ser avaliado.

b) Saída (escala de 0 a 10): reprovado (trapézio 0, 0, 3, 4,5), curso de férias (triângulo 3,5, 5, 6,5) e aprovado (trapézio 5,5, 7, 10, 10). As situações das pontas são trapézios porque qualquer valor muito baixo é reprovação e qualquer valor alto é aprovação. O curso de férias é uma situação intermediária, com um centro claro, por isso é um triângulo. Os conjuntos se cruzam em 4,0 e 6,0, e esses são os limites que usamos para dar a resposta final: abaixo de 4 reprovado, entre 4 e 6 curso de férias e acima de 6 aprovado.

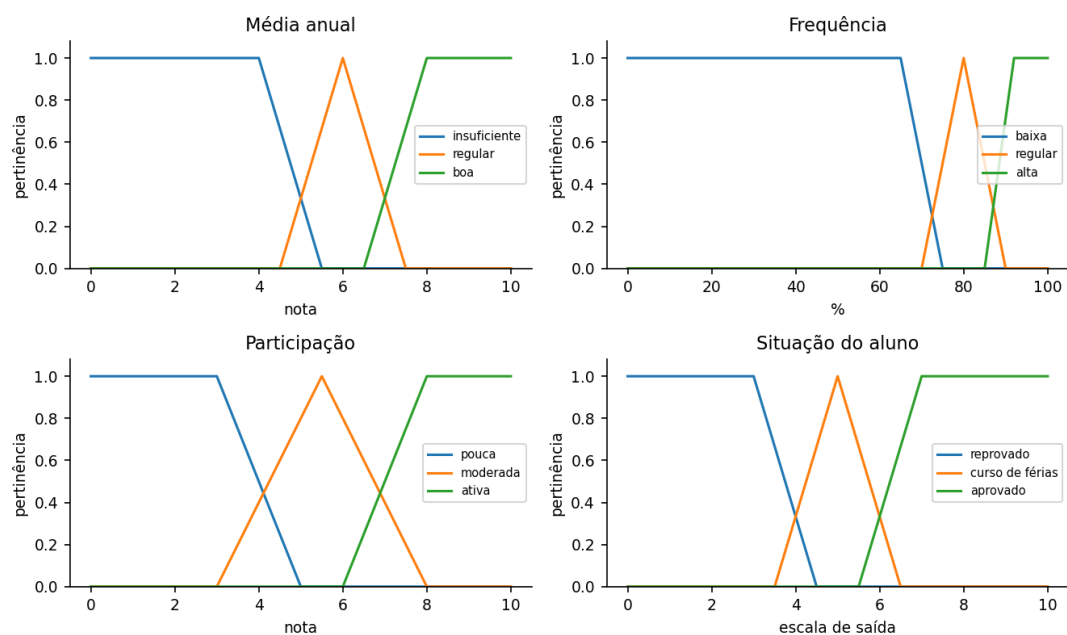


Figura 3. Conjuntos fuzzy das três entradas e da saída.

c) Regras.

- R1: Se a média é boa e a frequência não é baixa, então aprovado.
- R2: Se a média é regular e a frequência é alta, então aprovado.
- R3: Se a média é regular, a frequência é regular e a participação é ativa, então aprovado.
- R4: Se a média é regular, a frequência é regular e a participação é pouca ou moderada, então curso de férias.
- R5: Se a média é insuficiente, a frequência não é baixa e a participação é ativa, então curso de férias.
- R6: Se a média é insuficiente e a participação não é ativa, então reprovado.
- R7: Se a frequência é baixa, então reprovado.

A R7 mantém a regra de frequência mínima: mesmo com média boa, o aluno que falta demais reprova. A R5 é o caso que o sistema antigo não previa: o aluno com nota baixa mas que frequenta e participa ganha a chance do curso de férias.

d) Critério do consequente. Quando uma regra tem mais de uma condição ligada por E, o grau de disparo é o **mínimo** dos graus das condições (inferência de Mamdani). Para OU usamos o máximo e para NÃO usamos $1 - \mu$. O conjunto da saída é cortado na altura desse grau, e quando várias regras disparam os conjuntos cortados são juntados pelo máximo. O mínimo é a escolha mais conservadora: uma regra não pode valer mais que a sua condição mais fraca.

e) Defuzzificação. Usamos o **centroide** da região final,

$$y^* = \frac{\sum_y \mu(y) y}{\sum_y \mu(y)}.$$

Ele leva em conta todas as regras que dispararam, com o peso de cada uma, e por isso a resposta varia aos poucos quando uma entrada muda um pouco. O primeiro máximo seria mais simples, mas olharia só para a regra mais forte e voltaria a criar saltos.

Rodamos o sistema no script com alguns alunos:

Tabela 1. Resultados do sistema para alguns alunos.

Média	Frequência	Participação	Regras ativas	Saída
5,9	80%	9	R3 = 0,93	8,08 aprovado
6,0	80%	9	R3 = 1,00	8,10 aprovado
5,9	85%	7	R3 = 0,50, R4 = 0,40	7,06 aprovado
6,2	78%	4	R4 = 0,50	5,00 curso de férias
4,8	90%	8,5	R2 = 0,20, R5 = 0,47	6,29 aprovado
3,0	80%	2	R6 = 1,00	1,90 reprovado
8,0	60%	9	R7 = 1,00	1,90 reprovado

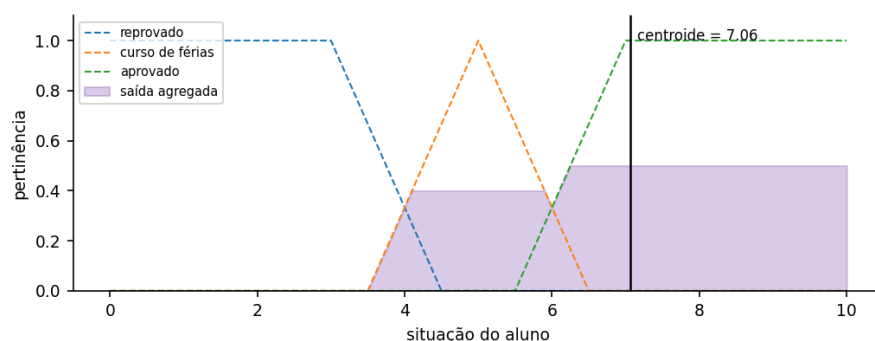


Figura 4. Aluno com média 5,9, frequência 85% e participação 7: R3 e R4 cortam “aprovado” e “curso de férias”, e o centroide cai em 7,06.

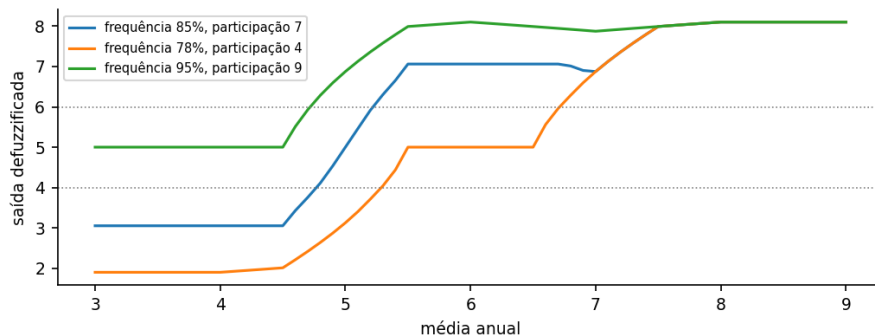


Figura 5. Saída em função da média, para três combinações de frequência e participação. As linhas pontilhadas são os limites 4 e 6.

Os alunos com 5,9 e 6,0 ficaram com 8,08 e 8,10, praticamente iguais, que era o objetivo. O aluno com 4,8 de média, 90% de frequência e participação 8,5 ficou em 6,29, no limite entre curso de férias e aprovação, porque frequência e participação compensaram a nota. A curva mostra que a saída sobe de forma contínua com a média, e a frequência e a participação deslocam a curva para cima ou para baixo.

Apêndice A. Código

```
pip install numpy matplotlib
python solucao.py
```

```
# Lista 3 - Lógica Fuzzy
# Sistemas Inteligentes Aplicados - UTFPR
# Uso: python solucao.py

import os
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

PASTA = os.path.dirname(os.path.abspath(__file__))
SAIDA = os.path.join(PASTA, "saidas")
os.makedirs(SAIDA, exist_ok=True)
plt.rcParams.update({"font.size": 9, "axes.spines.top": False,
                    "axes.spines.right": False})

def trap(x, a, b, c, d):
    # trapézio a-b-c-d; com a == b ou c == d vira ombro
    x = np.asarray(x, dtype=float)
    y = np.ones_like(x)
    if b > a:
        y = np.where(x < b, (x - a) / (b - a), y)
    if d > c:
        y = np.where(x > c, (d - x) / (d - c), y)
    y = np.where((x < a) | (x > d), 0, y)
    return np.clip(y, 0, 1)

def tri(x, a, b, c):
    return trap(x, a, b, b, c)

def grafico(ax, x, conjuntos, titulo, unidade):
    for nome, mu in conjuntos.items():
        ax.plot(x, mu(x), lw=1.5, label=nome)
    ax.set_title(titulo)
    ax.set_xlabel(unidade)
    ax.set_ylabel("pertinência")
    ax.set_ylim(0, 1.08)
    ax.legend(fontsize=7, loc="center right")

def mamdani(regras, entradas, saida_x, saida_conj):
    # cada regra: (grau de disparo calculado das entradas, conjunto da saída)
    agregado = np.zeros_like(saida_x)
    graus = []
    for grau_fn, conj in regras:
        g = float(grau_fn(entradas))
        graus.append(g)
        # corta o conjunto da saída no grau da regra e junta pelo máximo
        agregado = np.maximum(agregado, np.minimum(g, saida_conj[conj](saida_x)))
    if agregado.sum() == 0:
        return None, graus, agregado
    centroide = (agregado * saida_x).sum() / agregado.sum()
    return centroide, graus, agregado

# questao 1e
print("[1e] união e interseção")
u = list(range(10))
A = [0, 0, 0.1, 0.2, 0.3, 0.8, 0.9, 1, 1, 1]
B = [1, 1, 0.9, 0.8, 0.7, 0.5, 0.4, 0.2, 0.2, 0]
uniao = [max(a, b) for a, b in zip(A, B)]
inter = [min(a, b) for a, b in zip(A, B)]
print(" u      " + " ".join(f"{x:>4}" for x in u))
print(" A      " + " ".join(f"{x:>4}" for x in A))
print(" B      " + " ".join(f"{x:>4}" for x in B))
```

```

print(" A ou B " + " ".join(f"{x:>4}" for x in uniao))
print(" A e B " + " ".join(f"{x:>4}" for x in inter))

fig, ax = plt.subplots(figsize=(7, 2.8))
ax.plot(u, A, "o-", label="A", color="tab:blue")
ax.plot(u, B, "s-", label="B", color="tab:orange")
ax.fill_between(u, inter, color="tab:green", alpha=0.35, label="A e B (mínimo)")
ax.plot(u, uniao, "k--", lw=1, label="A ou B (máximo)")
ax.set_xticks(u)
ax.set_xlabel("u")
ax.set_ylabel("pertinência")
ax.legend(fontsize=7, ncol=2)
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q1e_uniao_intersecao.png"), dpi=150)
plt.close(fig)

# questao 2
print("\n[2] resort")
temp = {"fria": lambda x: trap(x, 0, 0, 10, 18),
        "agradável": lambda x: tri(x, 15, 23, 30),
        "quente": lambda x: trap(x, 27, 32, 40, 40)}
sol = {"nublado": lambda x: trap(x, 0, 0, 20, 40),
       "parcial": lambda x: tri(x, 30, 50, 70),
       "ensolarado": lambda x: trap(x, 60, 80, 100, 100)}
turistas = {"poucos": lambda x: trap(x, 0, 0, 20, 40),
            "médio": lambda x: tri(x, 30, 50, 70),
            "muitos": lambda x: trap(x, 60, 80, 100, 100)}

fig, ax = plt.subplots(1, 3, figsize=(11, 3))
grafico(ax[0], np.linspace(0, 40, 401), temp, "Temperatura", "°C")
grafico(ax[1], np.linspace(0, 100, 401), sol, "Luz do sol", "% do máximo")
grafico(ax[2], np.linspace(0, 100, 401), turistas, "Turistas",
        "% da capacidade")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q2_resort.png"), dpi=150)
plt.close(fig)

regras_resort = [
    (lambda e: min(temp["quente"](e[0]), sol["ensolarado"](e[1])), "muitos"),
    (lambda e: min(temp["agradável"](e[0]), sol["ensolarado"](e[1])), "muitos"),
    (lambda e: min(temp["agradável"](e[0]), sol["parcial"](e[1])), "médio"),
    (lambda e: temp["fria"](e[0]), "poucos"),
    (lambda e: sol["nublado"](e[1]), "poucos"),
]
y = np.linspace(0, 100, 1001)
for t, s in [(28, 65), (20, 45), (12, 80), (33, 90)]:
    c, graus, _ = mamdani(regras_resort, (t, s), y, turistas)
    g = " ".join(f"R{i + 1}={v:.2f}" for i, v in enumerate(graus))
    print(f" {t} °C e {s}% de sol: {g} -> {c:.1f}% da capacidade")

# questao 4
print("\n[4] situação do aluno")
media = {"insuficiente": lambda x: trap(x, 0, 0, 4, 5.5),
        "regular": lambda x: tri(x, 4.5, 6, 7.5),
        "boa": lambda x: trap(x, 6.5, 8, 10, 10)}
freq = {"baixa": lambda x: trap(x, 0, 0, 65, 75),
        "regular": lambda x: tri(x, 70, 80, 90),
        "alta": lambda x: trap(x, 85, 92, 100, 100)}
part = {"pouca": lambda x: trap(x, 0, 0, 3, 5),
        "moderada": lambda x: tri(x, 3, 5.5, 8),
        "ativa": lambda x: trap(x, 6, 8, 10, 10)}
situacao = {"reprovado": lambda x: trap(x, 0, 0, 3, 4.5),
            "curso de férias": lambda x: tri(x, 3.5, 5, 6.5),
            "aprovado": lambda x: trap(x, 5.5, 7, 10, 10)}

fig, ax = plt.subplots(2, 2, figsize=(9, 5.4))
grafico(ax[0, 0], np.linspace(0, 10, 401), media, "Média anual", "nota")
grafico(ax[0, 1], np.linspace(0, 100, 401), freq, "Frequência", "%")
grafico(ax[1, 0], np.linspace(0, 10, 401), part, "Participação", "nota")
grafico(ax[1, 1], np.linspace(0, 10, 401), situacao, "Situação do aluno",
        "escala de saída")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q4_conjuntos.png"), dpi=150)

```

```

plt.close(fig)

def nao(v):
    return 1 - v

# entradas: (média, frequência, participação)
regras_aluno = [
    (lambda e: min(media["boa"](e[0]), nao(freq["baixa"](e[1]))),
     "aprovado"),
    (lambda e: min(media["regular"](e[0]), freq["alta"](e[1])), "aprovado"),
    (lambda e: min(media["regular"](e[0]), freq["regular"](e[1]),
     part["ativa"](e[2])), "aprovado"),
    (lambda e: min(media["regular"](e[0]), freq["regular"](e[1]),
     max(part["pouca"](e[2]), part["moderada"](e[2]))),
     "curso de férias"),
    (lambda e: min(media["insuficiente"](e[0]), nao(freq["baixa"](e[1])),
     part["ativa"](e[2])), "curso de férias"),
    (lambda e: min(media["insuficiente"](e[0]),
     nao(part["ativa"](e[2]))), "reprovado"),
    (lambda e: freq["baixa"](e[1]), "reprovado"),
]

y = np.linspace(0, 10, 1001)

def avaliar(e):
    c, graus, agregado = mamdani(regras_aluno, e, y, situacao)
    graus_saida = {k: float(f(c)) for k, f in situacao.items()}
    return c, graus, agregado, max(graus_saida, key=graus_saida.get)

alunos = [(5.9, 80, 9), (6.0, 80, 9), (5.9, 85, 7), (8.5, 95, 6), (6.2, 78, 4),
           (4.8, 90, 8.5), (3.0, 80, 2), (8.0, 60, 9), (6.5, 88, 6)]
for e in alunos:
    c, graus, _, nome = avaliar(e)
    ativas = " ".join(f"R{i + 1}={g:.2f}" for i, g in enumerate(graus) if g > 0)
    print(f"  média {e[0]}, freq {e[1]}%, part {e[2]}: {ativas}"
          f"  -> {c:.2f} ({nome})")

# figura do primeiro aluno: regras cortadas e centroide
c, graus, agregado, nome = avaliar((5.9, 85, 7))
fig, ax = plt.subplots(figsize=(7, 2.8))
for k, f in situacao.items():
    ax.plot(y, f(y), lw=1, ls="--", label=k)
ax.fill_between(y, agregado, color="tab:purple", alpha=0.35,
               label="saída agregada")
ax.axvline(c, color="k", lw=1.2)
ax.text(c + 0.1, 1.0, f"centroide = {c:.2f}", fontsize=8)
ax.set_xlabel("situação do aluno")
ax.set_ylabel("pertinência")
ax.set_ylim(0, 1.1)
ax.legend(fontsize=7, loc="upper left")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q4_exemplo.png"), dpi=150)
plt.close(fig)

# a saída muda aos poucos com a média, sem o salto do 5,9 para o 6,0
notas = np.linspace(3, 9, 61)
fig, ax = plt.subplots(figsize=(7, 2.8))
for f_, p_ in [(85, 7), (78, 4), (95, 9)]:
    saidas = [avaliar((n, f_, p_))[0] for n in notas]
    ax.plot(notas, saidas, label=f"frequência {f_}%, participação {p_}")
# onde os conjuntos da saída se cruzam
ax.axhline(4.0, color="gray", lw=0.8, ls=":")
ax.axhline(6.0, color="gray", lw=0.8, ls=":")
ax.set_xlabel("média anual")
ax.set_ylabel("saída defuzzificada")
ax.legend(fontsize=7)
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q4_curva.png"), dpi=150)
plt.close(fig)

```